Understanding Pipelining Hazards

BLG 483E – Artificial Intelligence

salihsefer36

May 10, 2026

Istanbul Technical University

Generated with AI assistance, refined over 4 iterations.

Learning Objectives

After this lecture, you will be able to:

1. **Identify** the three classes of pipeline hazards — data, structural, and control — and give a concrete real-world analogy for each.
2. **Apply** forwarding and stall-insertion logic to a given instruction sequence, and calculate the resulting stall count and total cycle time.
3. **Compare** hardware mitigation strategies (forwarding, branch prediction, compiler scheduling) and explain which is most effective for each hazard class.

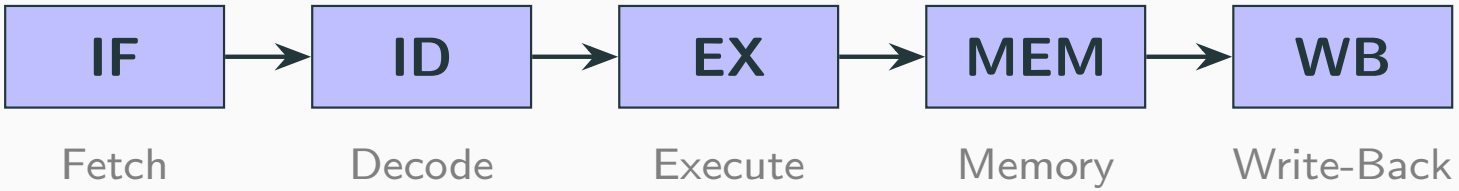
Outline

Introduction to CPU Pipelining

Think of a restaurant: one waiter takes orders, one chef cooks, another plates, another serves. If one person did everything per customer, the restaurant would serve one table at a time. A CPU pipeline works exactly the same way.

- A pipeline splits every instruction into 5 stages that run **simultaneously** for different instructions: **IF** (fetch), **ID** (decode), **EX** (execute), **MEM** (memory), **WB** (write result).
- When the assembly line runs smoothly, the CPU completes roughly one instruction per clock cycle (instead of one every 5 cycles).
- A **hazard** is anything that breaks the smooth flow — a conflict that forces a stage to wait or throw away work.
- Hazards come in three flavours: **data**, **structural**, and **control**.

The 5-Stage Pipeline



- **IF** fetches the raw binary instruction from the instruction cache.
- **ID** decodes opcodes and reads source-register values.
- **EX** runs the ALU operation (add, subtract, address compute).
- **MEM** reads from or writes to the data cache.
- **WB** commits the computed value to the destination register.

In a *hazard-free* pipeline a new instruction enters IF every cycle. **Hazards break this ideal.**

Data Hazards — Types

Imagine a relay race where runner B must grab the baton from runner A before sprinting. If B takes off before A arrives, the handoff fails. Data hazards are exactly that: one instruction needs a value the previous one has not finished computing yet.

- **RAW** (Read-After-Write, "true dependence"): the most common type. Instruction B tries to read a register before instruction A has written its result. *Runner B sprints before the baton arrives.*
- **WAR** (Write-After-Read, "anti-dependence"): B overwrites a register that A is still reading. *Chef B cleans the pan chef A is still using.*
- **WAW** (Write-After-Write, "output dependence"): two instructions write the same register; the second write must dominate. *Two workers file the same report — only the later one counts.*

Data Hazards — Detection & Mitigation

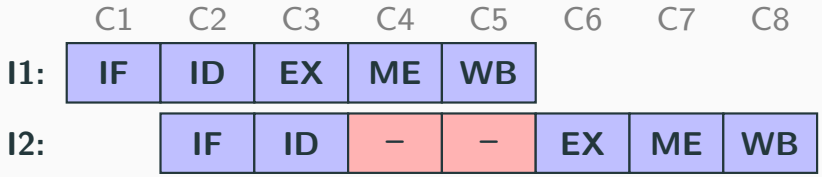
Think of a traffic officer watching an intersection. When two cars are about to collide, the officer either holds one back (stall) or waves it through a special shortcut lane (forwarding).

- **Detection:** A hazard detection unit checks the pipeline registers (ID/EX, EX/MEM, MEM/WB) every cycle to spot conflicts before they corrupt results.
- **Stall (interlock):** Freeze the fetch and decode stages and insert a bubble (a "do nothing" slot) — like a one-cycle red light.
- **Forwarding (bypassing):** Route the result directly from one stage to the next instruction's input, skipping the register file. Eliminates most RAW stalls completely.
- **Load-use exception:** A load followed immediately by a dependent instruction cannot be saved by forwarding — one stall is unavoidable.

Worked Example: RAW Hazard & Forwarding

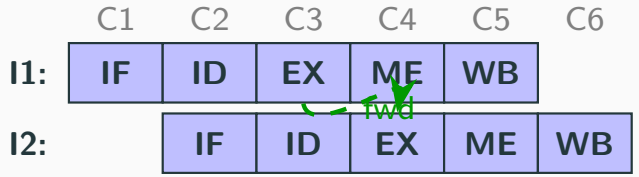
`I1: ADD R1,R2,R3` `I2: SUB R4,R1,R5` — I2 reads R1 produced by I1 (**RAW** dependence)

Without Forwarding



2 stall cycles ⇒ **8 cycles**

With Forwarding



0 stalls ⇒ **6 cycles**

Step-by-step (no fwd vs. fwd):

Instruction	C1	C2	C3	C4	C5	C6	C7	C8
I1 (ADD R1,R2,R3)	IF	ID	EX	ME	WB			
I2 without fwd		IF	ID	–	–	EX	ME	WB
I2 with fwd		IF	ID	EX	ME	WB		

Result: No fwd: **2 stalls** (8 cycles). Fwd: **0 stalls** (6 cycles) — **25% speedup**.

Structural Hazards

Two delivery trucks arriving at a warehouse with a single loading dock at the same time: one must wait. Structural hazards are the same idea — two pipeline stages need the same piece of hardware simultaneously.

- **Memory conflict:** Without separate instruction and data caches, the Fetch and Memory stages cannot both run in the same cycle.
- **Register write-port conflict:** A single-port register file cannot accept two write-back results simultaneously. *One printer, two jobs queued at once.*
- **Multi-cycle unit contention:** Slow FP operations (divide, square root) tie up a shared unit for many cycles.
- **Fixes:** Duplicate hardware, time-share the resource, or let the compiler insert independent work between the conflicting instructions.

Control Hazards

You are driving and your GPS says "turn left" — but you are already in the right lane three blocks back. A branch hazard is the same: the CPU grabs the wrong next instructions before it knows which way a branch goes.

- Branch outcome is resolved at EX, so 2 instructions are already fetched for the wrong path and must be **flushed**, wasting 2 cycles.
- **Static prediction:** Always guess "not taken". Simple, but wrong for loops.
- **Dynamic prediction (2-bit counter):** Learns from past behaviour — like a GPS that remembers your usual route.
- **Branch Target Buffer (BTB):** Caches the destination address so the CPU can jump immediately.
- **Delayed branch:** The compiler fills the post-branch slot with a useful instruction, recovering one cycle for free.

Hazard Mitigation Summary

A factory supervisor has three tools: a stop button (stall), a conveyor shortcut (forwarding), and a smart pre-planned schedule (compiler reordering). Together they keep the assembly line moving.

- **Stall:** Freeze the front of the pipeline and insert a bubble — buys time, but costs cycles.
- **Forwarding:** A hardware shortcut routing a result directly to the next instruction's input — cuts most RAW stalls to zero.
- **Load-use stall:** Forwarding cannot rescue this case; one wasted cycle is unavoidable.
- **Branch delay slot:** Filling the post-branch slot with real work (not `NOP`) recovers a cycle for free.
- **Compiler scheduling:** Static reordering eliminates many hazards before the hardware ever encounters them.

References

- Patterson, D. A. and Hennessy, J. L. *Computer Organization and Design: The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2014.
- Hennessy, J. L. and Patterson, D. A. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

Real-World Context

Pipelining hazards are not textbook abstractions — they determine the performance of every processor you interact with daily.

- **Modern x86 (Intel/AMD Core series):** 14–20+ pipeline stages; out-of-order execution and Tomasulo's algorithm handle RAW hazards dynamically via register renaming, eliminating most stalls at runtime. Branch predictors achieve $>98\%$ accuracy on typical workloads.
- **ARM Cortex family:** Cortex-A53 is a simple in-order pipeline where hazard avoidance is critical. Cortex-A76 is fully out-of-order with speculative execution — the same hazard concepts, radically different hardware solutions.
- **GPU SIMT / CUDA streams:** GPUs hide pipeline latency not with forwarding but with *warp switching* — when one warp stalls on memory, another runs instantly. Thousands of

Summary & Common Pitfalls

Key takeaways:

- **Three hazard classes:** data (RAW / WAR / WAW), structural (resource conflict), control (branch uncertainty).
- **Forwarding** eliminates most RAW stalls; the **load-use** case always costs exactly 1 cycle — no exception.
- **Dynamic branch prediction** (2-bit counter + BTB) cuts control-hazard penalties to near-zero for predictable code.

Common misconceptions:

- *"Forwarding solves all data hazards."* — False: a load followed by an immediately dependent instruction always stalls once.
- *"Deeper pipelines are always faster."* — False: deeper pipelines amplify branch penalties (e.g., Intel Pentium 4 Prescott, 31 stages, notorious for branch-miss performance).

Test Yourself